

SAE33 - Documentation Technique Windows Server

I - Accès SSH

Configuration avec clé privée publique

Sur le client

```
ssh-keygen -t ed25519
```

- Récupérer le contenu de la clé publique situé ici : `C:\Users\[UTILISATEUR ACTUEL]\.ssh\id_ed25519.pub`

Sur le serveur cible

Ce script installe le serveur SSH si ce n'est pas déjà le cas, active les bonnes règles de pare-feu, ajoute la clé publique du client et fait en sorte que la connexion en tant qu'administrateur soit possible.

```
$ErrorActionPreference = "Stop"

# Clé SSH publique à compléter ci-dessous
$PublicKey = "VOTRE CLÉ ICI"

# Variables
$AdminUser = "Administrateur"

# Installation du serveur SSH
$Cap = "OpenSSH.Server~~~~0.0.1.0"
if (!(Get-WindowsCapability -Online | Where-Object {$_.Name -like "$Cap" -and $_.State -eq 'I
nstalled'})) {
    Add-WindowsCapability -Online -Name $Cap | Out-Null
}

# Activation du service et configuration du pare-feu
Set-Service -Name sshd -StartupType 'Automatic'
Start-Service sshd

if (!(Get-NetFirewallRule -Name "OpenSSH-Server-In-TCP" -ErrorAction SilentlyContinue)) {
    New-NetFirewallRule -Name 'OpenSSH-Server-In-TCP' -DisplayName 'OpenSSH Server (ssh
d)' -Enabled True -Direction Inbound -Protocol TCP -Action Allow -LocalPort 22 | Out-Null
}

# Powershell par défaut
New-ItemProperty -Path "HKLM:\SOFTWARE\OpenSSH" -Name DefaultShell -Value "C:\Windo
ws\System32\WindowsPowerShell\v1.0\powershell.exe" -PropertyType String -Force | Out-Null
```

```

# Autoriser les clés dans le dossier .ssh de l'admin
$SshConfig = "C:\ProgramData\ssh\sshd_config"
$ConfContent = Get-Content $SshConfig
$NewConf = @()
$Skip = $false

foreach ($line in $ConfContent) {
    # Commente le bloc qui force l'admin à utiliser le fichier ProgramData
    if ($line -match "Match Group Administrators") {
        $NewConf += "# $line"
        $Skip = $true
    }
    elseif ($Skip -and ($line -match "AuthorizedKeysFile __PROGRAMDATA__")) {
        $NewConf += "# $line"
        $Skip = $false
    }
    else {
        $NewConf += $line
    }
}
Set-Content -Path $SshConfig -Value $NewConf -Encoding UTF8

# Clé publique client et permissions
$UserSshDir = "C:\Users\$AdminUser\.ssh"
$AuthKeyFile = "$UserSshDir\authorized_keys"

if (!(Test-Path $UserSshDir)) { New-Item -Path $UserSshDir -ItemType Directory -Force | Out-Null }

# Écriture de la clé
Set-Content -Path $AuthKeyFile -Value $PublicKey -Encoding Ascii

# Permissions ACL strictes (SYSTEM + Administrators uniquement)
$Acl = Get-Acl $AuthKeyFile
$Acl.SetAccessRuleProtection($true, $false) # Désactive héritage

$Rule_Sys = New-Object System.Security.AccessControl.FileSystemAccessRule("SYSTEM", "FullControl", "Allow")
$Rule_Adm = New-Object System.Security.AccessControl.FileSystemAccessRule(
    (New-Object System.Security.Principal.SecurityIdentifier("S-1-5-32-544")),
    "FullControl", "Allow"
)

$Acl.SetAccessRule($Rule_Sys)
$Acl.AddAccessRule($Rule_Adm)
Set-Acl -Path $AuthKeyFile -AclObject $Acl

```

```
# Redémarrage du service
Restart-Service sshd
```

Ajout d'une machine supplémentaire

Ce script permet d'ajouter une nouvelle machine client à la liste des clés publiques autorisées. Il suffit de coller la clé publique avant d'exécuter.

```
# Variables
# Remplacez le texte ci-dessous par votre clé publique
$NewKey = "VOTRE CLÉ ICI"

$KeyFile = "C:\Users\Administrateur\.ssh\authorized_keys"

$NewKey = $NewKey.Trim()

# Vérification que la clé n'est pas vide
if ([string]::IsNullOrEmpty($NewKey) -or $NewKey -like "*COPIEZ_VOTRE_CLE_ICI*") {
    Write-Error "Erreur : La variable `$NewKey n'a pas été modifiée."
    exit
}

# Vérification que le fichier existe
if (!(Test-Path $KeyFile)) {
    Write-Error "Erreur : Le fichier authorized_keys est introuvable."
    exit
}

# Vérification doublon et ajout
$CurrentContent = Get-Content $KeyFile
if ($CurrentContent -contains $NewKey) {
    Write-Host "Cette clé existe déjà dans le fichier." -ForegroundColor Yellow
}
else {
    Add-Content -Path $KeyFile -Value $NewKey -Encoding Ascii
    Write-Host "Succès : Nouvelle clé ajoutée." -ForegroundColor Green
}
```

II - Domaine Active Directory

Ce script peuple le domaine en suivant la méthode AGDLP et applique les GPOs demandées.

```
$ErrorActionPreference = "Stop"

# Variables
$DomainName = "G7-Pellet.com.test"
$RootDN = "DC=G7-Pellet,DC=com,DC=test"
```

```

$ShareLocalPath = "C:\Partages\Documents Administratifs"
$ShareName      = "Documents Administratifs"
$ShareUNC       = "\\$DomainName\$ShareName"
$DriveLetter    = "Z"
$WallpaperUNC   = "\\$DomainName\NETLOGON\feu-de-camp.jpg"
$GPOName        = "GPO_Client_Standard"

# Arborescence
$RootOU = "OU=G7-Pellet,$RootDN"

if (!(Get-ADOrganizationalUnit -Filter "Name -eq 'G7-Pellet'")) {
    New-ADOrganizationalUnit -Name "G7-Pellet" -Path $RootDN
}

foreach ($ou in @("Utilisateurs", "Groupes", "Ordinateurs")) {
    if (!(Get-ADOrganizationalUnit -Filter "Name -eq '$ou'" -SearchBase $RootOU)) {
        New-ADOrganizationalUnit -Name $ou -Path $RootOU
    }
}

# Groupes et utilisateurs (méthode AGDLP)

# Groupes
$Groups = @(
    @{Name="GG_Service_Admin"; Scope="Global"; Path="OU=Groupes,$RootOU"},
    @{Name="DLG_DocsAdmin_RW"; Scope="DomainLocal"; Path="OU=Groupes,$RootOU"},
    @{Name="DLG_DocsAdmin_RO"; Scope="DomainLocal"; Path="OU=Groupes,$RootOU"}
)

foreach ($grp in $Groups) {
    if (!(Get-ADGroup -Filter "Name -eq '$($grp.Name)'")) {
        New-ADGroup -Name $grp.Name -GroupScope $grp.Scope -Path $grp.Path
    }
}

# Imbrication des groupes
Add-ADGroupMember -Identity "DLG_DocsAdmin_RW" -Members "GG_Service_Admin" -Error
Action SilentlyContinue
Add-ADGroupMember -Identity "DLG_DocsAdmin_RO" -Members "Utilisateurs du domaine" -E
rrorAction SilentlyContinue

# Utilisateur Florian pour les tests
$User = "fvigneux"
if (!(Get-ADUser -Filter "SamAccountName -eq '$User'")) {
    $Pass = ConvertTo-SecureString "Password123!" -AsPlainText -Force
    New-ADUser -Name "Florian Vigneux" -SamAccountName $User -UserPrincipalName "$Us
er@$DomainName" `
        -AccountPassword $Pass -Enabled $true -Path "OU=Utilisateurs,$RootOU" `

```

```

-ChangePasswordAtLogon $false

Add-ADGroupMember -Identity "GG_Service_Admin" -Members $User
}

# Partage SMB et permissions NTFS

if (!(Test-Path $ShareLocalPath)) { New-Item -ItemType Directory -Path $ShareLocalPath | Out-Null }

# Partage SMB
if (!(Get-SmbShare -Name $ShareName -ErrorAction SilentlyContinue)) {
    New-SmbShare -Name $ShareName -Path $ShareLocalPath -FullAccess "Tout le monde" | Out-Null
}

# ACL NTFS
$Acl = Get-Acl $ShareLocalPath
$Acl.SetAccessRuleProtection($true, $false) # Désactiver héritage

$Rule_Admin = New-Object System.Security.AccessControl.FileSystemAccessRule("Administrateurs", "FullControl", "ContainerInherit,ObjectInherit", "None", "Allow")
$Rule_RW = New-Object System.Security.AccessControl.FileSystemAccessRule("$DomainName\DLG_DocsAdmin_RW", "Modify", "ContainerInherit,ObjectInherit", "None", "Allow")
$Rule_RO = New-Object System.Security.AccessControl.FileSystemAccessRule("$DomainName\DLG_DocsAdmin_RO", "ReadAndExecute", "ContainerInherit,ObjectInherit", "None", "Allow")

$Acl.SetAccessRule($Rule_Admin)
$Acl.AddAccessRule($Rule_RW)
$Acl.AddAccessRule($Rule_RO)
Set-Acl -Path $ShareLocalPath -AclObject $Acl

# Création de la GPO
if (!(Get-GPO -Name $GPOName -ErrorAction SilentlyContinue)) {
    New-GPO -Name $GPOName | Out-Null
}
# Liaison à la racine du domaine
$GPO = Get-GPO -Name $GPOName
try {
    New-GPLink -Name $GPOName -Target $RootDN -LinkEnabled Yes -ErrorAction Stop | Out-Null
} catch {
    # Ignore si déjà lié
}

# Fond d'écran feu de camp via le registre
Set-GPRegistryValue -Name $GPOName -Key "HKCU\Control Panel\Desktop" -ValueName "Wallpaper" -Type String -Value $WallpaperUNC

```

```
Set-GPRegistryValue -Name $GPOName -Key "HKCU\Control Panel\Desktop" -ValueName "WallpaperStyle" -Type String -Value "10"
```

```
# Configuration du lecteur Z: pointant vers le partage SMB
```

```
$GPOID = $GPO.Id
```

```
$SysvolPolicies = "\\$DomainName\SYSVOL\$DomainName\Policies\{$GPOID}"
```

```
$SysvolDrives = "$SysvolPolicies\User\Preferences\Drives"
```

```
if (!(Test-Path $SysvolDrives)) { New-Item -Path $SysvolDrives -ItemType Directory -Force | Out-Null }
```

```
$XmlContent = @"
```

```
<?xml version="1.0" encoding="utf-8"?>
```

```
<Drives clsid="{8FDDCC1A-0C3C-43cd-A6B4-71A6DF20DA8C}">
```

```
  <Drive clsid="{935F44F5-8579-4886-999E-7756F69E3A57}" name="$DriveLetter`:" status="$DriveLetter`:" image="2" changed="2024-01-01 12:00:00" uid="{${[guid]::NewGuid()}}">
```

```
    <Properties action="U" thisDrive="NOCHANGE" allDrives="NOCHANGE" userName="" path="$ShareUNC" label="$ShareName" persistent="1" useLetter="1" letter="$DriveLetter"/>
```

```
  </Drive>
```

```
</Drives>
```

```
"@
```

```
Set-Content -Path "$SysvolDrives\Drives.xml" -Value $XmlContent -Encoding UTF8
```

```
# Mise à jour de la version de la GPO
```

```
$GPTIni = "$SysvolPolicies\GPT.INI"
```

```
$Content = Get-Content $GPTIni
```

```
$NewContent = @()
```

```
foreach ($line in $Content) {
```

```
  if ($line -match "Version=(\d+)") {
```

```
    # On ajoute 100
```